گزارش پروژه‌ی درس رایانش ابری

مستندات کامل ربات تلگرام مدیریت کارهای روزانه

taskify — Serverless Telegram Task Bot on Cloudflare

نام دانشجو	**سجاد اسماعیلی**
استاد راهنما	**امیر آبرون**
نام درس	**رایانش ابری**
تاریخ	**آبان ۱۴۰۵**

پیاده‌سازی با Cloudflare Workers، D1 و Workers AI

۱. چکیده و مقدمه

taskify یک ربات تلگرام فارسی برای مدیریت کارهای روزانه است که کاملاً سرورلس روی Cloudflare اجرا می‌شود. کاربر کارهایش را با جمله‌های محاوره‌ای می‌نویسد (مثلاً «فردا برم مهمونی» یا «۱۰ دقیقه بعد یادم بنداز برم باشگاه»)؛ یک مدل زبانی (Llama 3.3 70B روی Workers AI) منظور او را به یک تصمیم JSON تبدیل می‌کند و کد، پس از اعتبارسنجی، آن را روی پایگاه داده‌ی D1 اجرا می‌کند. مدل هرگز مستقیم به داده دست نمی‌زند. کل منطق در یک فایل Worker قرار دارد.

اهداف پروژه

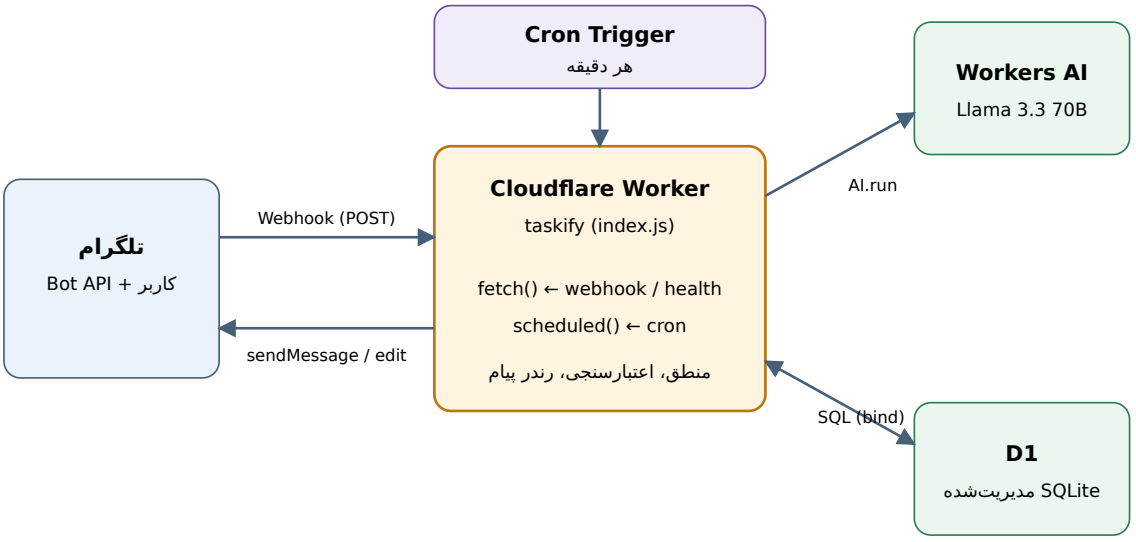
- اجرای کامل روی بستر ابری، بدون راه‌اندازی یا نگه‌داری سرور.
- فهم زبان طبیعی فارسی همراه با ایمنی نوشتن داده (اعتبارسنجی خروجی مدل).
- نگه‌داری وضعیت در پایگاه داده‌ی مدیریت‌شده و اجرای کارهای زمان‌بندی‌شده (پیام صبح، یادآوری دقیق).
- امنیت دسترسی: احراز اصالت وبهوک، فهرست سفید کاربران و مدیریت امن اسرار.

ارتباط با مفاهیم رایانش ابری

مفهوم	نحوه‌ی تحقق در پروژه
سرورلس / FaaS	کل برنامه یک `Cloudflare Worker` است؛ هر درخواست یک اجرای مستقل و مقیاس‌پذیری خودکار.
معماری رویدادمحور	دو منبع رویداد: وبهوک HTTP تلگرام و Cron Trigger هر دقیقه.
پایگاه داده به‌عنوان سرویس	D1 (SQLite مدیریت‌شده) با binding به نام `DB`.
هوش مصنوعی به‌عنوان سرویس	Workers AI با binding به نام `AI`؛ بدون راه‌اندازی یا مدیریت GPU.
زیرساخت به‌صورت کد و اسرار	پیکربندی در wrangler؛ توکن و کلید وبهوک با `wrangler secret`.
مشاهده‌پذیری	فعال‌بودن observability و مشاهده‌ی زنده‌ی لاگ با tail.
قابلیت اطمینان	پاسخ همیشه‌ی ۲۰۰ برای وبهوک، «claim» اتمی برای ارسال یادآوری و تلاش مجدد محدود.

۲. معماری سیستم

یک Worker در مرکز قرار دارد و با سه سرویس پیرامونی کار می‌کند: Workers AI برای فهم زبان، D1 برای ذخیره‌سازی و Telegram Bot API برای ارتباط با کاربر. تریگر cron هر دقیقه تابع `scheduled()` را فرا می‌خواند.



شکل ۱. معماری کلی سامانه

مسیر پردازش یک پیام

۱. تلگرام به `/webhook` درخواست POST می‌فرستد؛ هدر `X-Telegram-Bot-Api-Secret-Token` با مقایسه‌ی زمان‌ثابت بررسی می‌شود (نادرست → ۴۰۳).
۲. پاسخ ۲۰۰ فوراً برمی‌گردد و پردازش با `ctx.waitUntil` ادامه می‌یابد تا تلاش مجدد تلگرام رخ ندهد.
۳. فشردن دکمه‌های اینلاین (`callback_query`)، دستور `/start` و فهرست سفید کاربران بررسی می‌شوند؛ پیام کاربر غیرمجاز نادیده گرفته می‌شود.
۴. سه دکمه‌ی کیبورد بدون فراخوانی مدل پاسخ داده می‌شوند.
۵. در غیر این صورت `decide()` پرامپت (لیست کارها، جدول روزها، ساعت فعلی، ۸ پیام آخر) را به مدل می‌دهد.
۶. `validate()` خروجی را بررسی می‌کند (حداکثر دو تلاش؛ در شکست، پیام راهنمای ثابت).
۷. `apply()` روی D1 می‌نویسد و پاسخ یا کارت لیست را به تلگرام می‌فرستد.

اجزای ابری

جزء	نقش
Cloudflare Workers	اجرای منطق؛ `fetch()` برای `/webhook` و `/health`، `scheduled()` برای cron
Workers AI / D1	ذخیره‌ی کارها، حافظه و یادآوری‌ها / تبدیل جمله‌ی کاربر به تصمیم JSON
Cron Triggers	پیام ۵ صبح، یادآوری ۸ صبح و ارسال یادآوری‌های سررسیدشده (عبارت `* * * * *`)
Secrets / Vars	توکن ربات و کلید وبهوک / `ALLOWED_USER_ID`، `LLM_MODEL`

۳. قابلیت‌ها

قابلیت	توضیح
مدیریت کارها	افزودن، تیک‌زدن، برگرداندن، حذف و نمایش با زبان طبیعی؛ عبارت «همه» برای همه‌ی کارهای یک روز.
روزهای آینده	روز کار با `days_ahead` (تا ۳۶۵ روز)؛ جدول ۱۴ روز آینده به مدل داده می‌شود؛ برچسب «(فردا)» و «(N روز دیگه)».
کارت لیست	شمارنده، نماد ⏳/✅ و برای هر کار دکمه‌ی تیک و 🗑️؛ فشردن دکمه همان پیام را در جا به‌روز می‌کند؛ حداکثر ۲۵ کار.
کنترل دسترسی	`ALLOWED_USER_ID` (یک یا چند آیدی). `/start`: کاربر مجاز خوش‌آمد می‌شنود؛ دیگران پیام رد با آیدی عددی خودشان و `@X_triples` می‌بینند.
کیبورد دائمی	سه دکمه: «لیست کارهای امروز»، «مناسبت‌های امروز» و «حمایت مالی» (لینک `https://www.coffeete.ir/sajjad_es`)؛ بدون مدل پاسخ داده می‌شوند.
مناسبت‌های جهانی	۱۱۶ مناسبت با تاریخ ثابت (ولنتاین، هالووین، روز سگ و ...) و ۹ قاعده‌ی متحرک (روز مادر، شکرگزاری، عید پاک و ...).
پیام ۵ صبح	هر روز برای کاربران مجاز: سلام، «صبح بخیر»، روز هفته، تاریخ شمسی و میلادی.
یادآوری ۸ صبح	کارت لیست امروز، فقط اگر برای امروز کاری ثبت شده باشد.
یادآوری زمان‌دار	«۱۰ دقیقه بعد یادم بنداز ...» یا «فردا ساعت ۵ یادم بنداز ...»؛ ارسال سر ساعت با cron هر دقیقه؛ دکمه‌ی «لغو یادآوری».



شکل ۲. نمونه‌ی کارت لیست با دکمه‌های اینلاین

۴. هوش مصنوعی، داده و امنیت

لایه‌ی هوش مصنوعی

- مدل `@cf/meta/llama-3.3-70b-instruct-fp8-fast` از راه binding بومی `AI`؛ دمای ۰٫۲ و در تلاش دوم ۰.
- قرارداد: یک شیء JSON با هفت کلید (`action, task_id, text, days_ahead, time, minutes_ahead, reply`) و هشت عمل (`add, done, undone, del, list, events, remind, chat`).
- اعتبارسنجی: استخراج JSON، بررسی ساختار و بازه‌ها، تطبیق `task_id` با ردیف واقعی، دو تلاش و سپس پیام راهنما؛ متن تأیید همیشه با کد ساخته می‌شود، نه مدل.
- مدل تاریخ حساب نمی‌کند؛ فقط شماره‌ی روز را از جدول انتخاب می‌کند و کد تاریخ واقعی را می‌سازد.
- حافظه‌ی گفتگو: ۸ پیام آخر دیده می‌شود و برای هر کاربر ۴۰ ردیف نگه داشته می‌شود.

مدل داده (D1)

جدول	ستون‌های اصلی	نقش
`tasks`	`id, user_id, day, title, done, created_at, done_at`	کارها؛ `day` به‌صورت `YYYY-MM-DD` به وقت تهران
`messages`	`id, user_id, role, content, created_at`	حافظه‌ی گفتگو
`reminders`	`id, user_id, due_at, title, sent, attempts, created_at`	یادآوری‌ها؛ `due_at` میلی‌ثانیه‌ی UTC؛ خودکار ساخته می‌شود

همه‌ی پرس‌وجوها شرط `user_id` دارند و با `prepare().bind()` نوشته شده‌اند؛ پس هر کاربر فقط داده‌ی خودش را می‌بیند.

امنیت

تهدید	راهکار
جعل درخواست وبهوک	`secret_token` تلگرام و مقایسه‌ی زمان‌ثابت؛ در ناهمخوانی ۴۰۳ بدون توضیح.
کاربر غیرمجاز	فهرست سفید آیدی عددی؛ پیام‌ها و دکمه‌های او نادیده گرفته می‌شود.
افشای اسرار	توکن و کلید وبهوک فقط با `wrangler secret`، نه در کد یا پیکربندی.
خروجی نادرست مدل	اعتبارسنجی ساختاری و معنایی؛ در شکست هیچ نوشتنی انجام نمی‌شود.
دسترسی به داده‌ی دیگران	`user_id` در همه‌ی پرس‌وجوها و بررسی مالکیت در هر فشردن دکمه.
ارسال تکراری یادآوری	«claim» اتمی با `UPDATE ... WHERE sent = 0` و حداکثر ۵ تلاش.

۵. استقرار، آزمون و جمع‌بندی

استقرار

```
npx wrangler d1 create hermes-mini-2
npx wrangler d1 migrations apply hermes-mini-2 --remote
npx wrangler secret put TELEGRAM_BOT_TOKEN
npx wrangler secret put TELEGRAM_WEBHOOK_SECRET
npx wrangler deploy
curl "https://api.telegram.org/bot<TOKEN>/setWebhook" -d "url=https://<worker>/webhook"
\
  -d "secret_token=<SECRET>" -d 'allowed_updates=["message","callback_query"]'
```

- در wrangler: `"crons": ["* * * * *"]` (هر دقیقه، برای یادآوری‌های دقیق)، متغیرهای `ALLOWED_USER_ID` و `LLM_MODEL`، و bindingهای `DB` و `AI`.
- سلامت سرویس: `GET /health` با یک SELECT 1 وضعیت D1 را نشان می‌دهد؛ لاگ زنده با `npm run tail`.

آزمون

آزمون‌های خودکار در Node.js کل Worker را با پایگاه داده‌ی SQLite شبیه‌ساز D1، مدل برنامه‌ریزی‌شده، تلگرام ساختگی و ساعت شبیه‌سازی‌شده اجرا کردند: کنترل دسترسی، کارت و دکمه‌ها، روزهای آینده، اعتبارسنجی خروجی مدل، پیام ۵ صبح، مناسبت‌ها (ثابت و متحرک) و یادآوری‌ها (گرد کردن، ارسال دقیق، نبود ارسال تکراری، تلاش مجدد، لغو). اتصال واقعی به تلگرام و مدل با چک‌لیست دستی (دکمه‌ها، `/start`، یادآوری یک‌دقیقه‌ای، `/health`) بررسی می‌شود.

محدودیت‌ها و توسعه‌ی آینده

- دقت یادآوری یک دقیقه است؛ تشخیص صبح/عصر برای «ساعت ۵» قاعده‌ای تجربی دارد و ساعت نهایی در پیام تأیید نمایش داده می‌شود.
- فهرست مناسبت‌ها ایستا است و مناسبت‌های قمری و ایرانی ندارد؛ نمایش یا لغو یادآوری با متن آزاد هنوز نیست.
- پیشنهاد: لیست و تکرار یادآوری، تقویم ایرانی، منطقه‌ی زمانی هر کاربر و آزمون‌های خودکار در خط لوله‌ی CI.

جمع‌بندی

این پروژه نشان می‌دهد با ترکیب `Workers`، D1 و Workers AI می‌توان یک دستیار کامل، امن و کم‌هزینه را بدون هیچ سرور اختصاصی ساخت. نکته‌ی کلیدی طراحی، جداکردن «تصمیم» (مدل) از «اجرا» (کد اعتبارسنجی‌شده) است.